CN-Stats-Lec07-Probability Distribution & Central Limit Theorem & Sampling Distributions

Probability Distribution & Central Limit Theorem & Sampling Distributions

Session Objectives:

- ✅ Discrete (PMF): compute simple probabilities.
- ✅ Continuous: plot & read histograms with Matplotlib.
- ✅ Distributions: Normal (μ,σ) | Standard Normal (0,1) | Poisson (λ).
- ✅ Simulation: show sample mean → true mean as n grows.
- ✅ Define CLT and know when it applies across different data types.
- 🔁 Connect to simulations you've run and apply CLT in practice.
- 🧠 Why it matters: use CLT for inference/normal approximations in real analyses.
- 📊 Sampling distributions: build them, describe their shape, and see how n ↑ ⇒ shape ~ normal.
- 🧮 Compute sampling mean & standard error; understand SE = σ/√n and its link to SD under CLT.

```python
# Coin Toss Probability Distribution
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random

coin_flips = [random.choice(['H','T']) for _ in range(10000)]
print(coin_flips)
```

```
['T', 'H', 'H', 'H', 'T', 'H', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'H',
'T', 'T', 'T', 'T', 'H', 'H', 'H', 'T', 'T', 'T', 'T', 'T', 'H', 'H', 'H', 'T', 'T', 'H', 'H', 'T', 'T',
'T', 'H', 'T', 'H', 'H', 'H', 'H', 'H', 'T', 'H', 'T', 'H', 'T', 'T', 'H', 'T', 'H', 'H', 'T', 'H', 'H',
'T', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'T', 'H', 'T', 'H', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'H', 'T',
'H', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'H', 'H', 'T', 'T', 'T', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'H',
'H', 'H', 'H', 'T', 'H', 'H', 'T', 'T', 'H', 'H', 'H', 'T', 'T', 'T', 'H', 'H', 'H', 'H', 'H', 'H', 'H',
'T', 'T', 'T', 'H', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'T', 'T', 'H', 'H', 'T', 'H', 'T', 'H', 'T', 'T',
'T', 'T', 'H', 'T', 'H', 'H', 'H', 'T', 'H', 'T', 'T', 'H', 'H', 'H', 'T', 'T', 'T', 'H', 'H', 'T', 'T',
'T', 'T', 'H', 'H', 'H', 'T', 'H', 'T', 'T', 'H', 'T', 'T', 'H', 'T', 'T', 'T', 'H', 'T', 'T', 'H', 'T',
'H', 'H', 'H', 'H', 'H', 'H', 'H', 'T', 'H', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'T', 'T', 'H', 'H', 'T',
'H', 'T', 'T', 'T', 'H', 'H', 'H', 'T', 'H', 'T', 'H', 'T', 'T', 'T', 'T', 'T', 'H', 'H', 'T', 'T', 'T',
'H', 'H', 'H', 'T', 'T', 'T', 'T', 'T', 'H', 'T', 'H', 'H', 'H', 'H', 'T', 'H', 'H', 'T', 'H', 'T', 'T',
'H', 'T', 'T', 'T', 'T', 'H', 'T', 'T', 'T', 'H', 'T', 'H', 'T', 'T', 'H', 'T', 'T', 'T', 'T', 'H', 'H',
'T', 'H', 'T', 'T', 'T', 'T', 'T', 'H', 'H', 'H', 'T', 'T', 'H', 'T', 'H', 'H', 'H', 'H', 'H', 'H', 'T',
'H', 'T', 'H', 'H', 'H', 'H', 'T', 'T', 'T', 'T', 'T', 'T', 'H', 'T', 'H', 'T', 'T', 'T', 'H', 'T', 'T',
'H', 'T', 'T', 'H', 'T', 'H', 'H', 'H', 'H', 'T', 'H', 'T', 'T', 'T', 'H', 'H', 'T', 'T', 'T', 'H', 'H',
'T', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'T', 'H', 'T', 'T', 'T', 'H', 'T', 'T', 'H', 'T', 'T', 'T', 'T',
```

```python
head_count = 0
tail_count = 0
for coin in coin_flips:
    if coin == 'H': # head
        head_count +=1
    else: # tail
        tail_count +=1

print(head_count)
print(tail_count)
```

```
5009
4991
```

```python
# Probability Distribution
import random
import collections
dir(collections)
```

```
['ChainMap',
 'Counter',
 'OrderedDict',
 'UserDict',
 'UserList',
 'UserString',
 '_Link',
 '_OrderedDictItemsView',
 '_OrderedDictKeysView',
 '_OrderedDictValuesView',
 '__all__',
 '__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '  name  ',
```

```python
# Coin Toss Probability Distributions
trails = 10000
result = []

for _ in range(trails):
    outcome = random.choice(['H','T'])
    if outcome == 'H':
        result.append(1) # +$1 Win
    else:
        result.append(-1) # -$1 Loss

print(result)
```

```
[-1, -1, 1, 1, 1, 1, -1, 1, 1, -1, 1, 1, 1, 1, -1, -1, 1, 1, -1, -1, 1, -1, 1, -1, -1, 1, -1, -1, -1, 1, 1,
-1, -1, 1, -1, -1, 1, -1, 1, -1, 1, 1, 1, 1, 1, 1, 1, 1, 1, -1, -1, -1, 1, 1, 1, -1, 1, -1, 1, 1, 1, -1, 1,
-1, 1, -1, 1, -1, 1, -1, -1, -1, 1, 1, 1, -1, -1, -1, -1, -1, 1, -1, 1, 1, 1, 1, 1, 1, -1, 1, 1, 1, 1, -1, -
1, 1, -1, 1, -1, -1, -1, -1, 1, -1, -1, 1, -1, -1, -1, -1, 1, -1, -1, -1, 1, 1, 1, 1, -1, -1, -1, -1, -1, -
1, 1, -1, -1, -1, -1, -1, 1, 1, -1, -1, -1, -1, 1, -1, 1, -1, -1, -1, -1, 1, 1, -1, -1, 1, -1, 1, -1, -1, -
1, -1, -1, -1, -1, 1, 1, -1, -1, 1, -1, -1, -1, 1, 1, -1, -1, 1, 1, -1, 1, -1, -1, -1, 1, -1, -1, 1, 1, -1,
1, -1, 1, -1, 1, -1, -1, -1, 1, 1, 1, 1, 1, 1, -1, 1, 1, -1, 1, 1, -1, -1, -1, 1, 1, 1, 1, -1, 1, 1, 1, 1, -
1, 1, 1, -1, -1, -1, -1, -1, -1, 1, -1, -1, 1, 1, -1, 1, 1, -1, 1, -1, 1, -1, 1, -1, 1, 1, -1, 1, -1, 1, 1,
-1, 1, -1, -1, 1, -1, 1, 1, -1, 1, 1, 1, -1, 1, -1, 1, 1, 1, 1, 1, 1, 1, 1, -1, -1, -1, -1, 1, 1, -1, 1, -1,
1, -1, 1, -1, 1, 1, -1, -1, 1, 1, -1, -1, 1, -1, -1, -1, 1, -1, 1, -1, 1, 1, -1, -1, 1, -1, 1, 1, -1, 1, -1,
-1, -1, 1, -1, -1, -1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, -1, 1, -1, -1, 1, -1, 1, -1, -1, 1, 1, -1, -1, 1, 1, -
1, -1, -1, -1, -1, -1, 1, -1, -1, -1, 1, 1, -1, -1, -1, -1, 1, -1, -1, 1, -1, -1, 1, -1, -1, 1, -1, -1, -1,
-1, 1, -1, -1, 1, 1, 1, -1, -1, 1, -1, 1, -1, 1, 1, -1, 1, 1, -1, -1, 1, 1, 1, 1, 1, -1, -1, 1, 1, 1, -1, -
1, -1, -1, -1, -1, -1, 1, -1, -1, -1, -1, 1, 1, 1, -1, 1, 1, 1, -1, 1, 1, -1, 1, -1, 1, -1, 1, 1, -1, -1, -
1, 1, -1, 1, -1, -1, -1, -1, -1, 1, 1, -1, 1, 1, -1, -1, -1, 1, 1, 1, -1, -1, 1, 1, 1, -1, -1, 1, -1, -1, 1,
1, 1, -1, 1, -1, -1, 1, 1, 1, 1, -1, 1, 1, 1, 1, 1, -1, -1, 1, 1, 1, -1, -1, -1, 1, -1, -1, 1, 1, -1, 1, -1,
1, -1, 1, -1, -1, -1, -1, -1, 1, 1, -1, -1, -1, 1, 1, -1, 1, -1, -1, 1, -1, -1, -1, 1, 1, 1, 1, 1, 1, -1, -
```

```python
sum(result)
```

```
-100
```

```python
from collections import Counter
counter = Counter(result) # Dict
counter
```

```
Counter({-1: 5050, 1: 4950})
```

```python
counter.items() # (key,value)
```

```
dict_items([(-1, 5050), (1, 4950)])
```

```python
for coin,freq in counter.items():
    print(f"{coin} : {freq/trails}")
```

```
-1 : 0.505
1 : 0.495
```

```python
# Die Roll Probability Distribution
trails = 10000
result = []

for _ in range(trails):
    outcome = random.randint(1,6) # [1,2,3,4,5,6]
    if outcome in [1,2,3]:
        result.append(2) # '$2 Win'
    else : # [4,5,6]
        result.append(5) # '$5 Win'

print(result)
```

```
[2, 5, 5, 5, 5, 2, 2, 5, 2, 2, 5, 5, 5, 5, 5, 5, 2, 2, 5, 5, 2, 5, 2, 2, 2, 2, 5, 5, 2, 5, 5, 2, 5, 2, 2, 5,
5, 2, 2, 5, 5, 5, 2, 2, 5, 5, 2, 5, 5, 5, 5, 2, 5, 5, 5, 5, 2, 5, 5, 5, 5, 5, 5, 5, 5, 2, 2, 5, 2, 5, 2, 2,
2, 5, 5, 2, 2, 2, 5, 5, 2, 2, 5, 5, 2, 2, 5, 2, 2, 5, 2, 2, 2, 2, 2, 5, 2, 2, 2, 5, 5, 2, 2, 5, 5, 5, 2, 5,
5, 2, 5, 2, 2, 5, 2, 2, 2, 2, 5, 5, 2, 2, 2, 2, 5, 5, 5, 2, 2, 2, 2, 5, 2, 2, 5, 5, 5, 2, 2, 2, 5, 2, 2, 5,
5, 5, 2, 2, 5, 2, 5, 2, 5, 2, 5, 2, 2, 5, 2, 2, 2, 5, 5, 2, 5, 2, 2, 2, 2, 5, 5, 5, 5, 2, 5, 5, 5, 5, 2, 5,
2, 5, 2, 5, 5, 2, 5, 5, 5, 2, 5, 2, 2, 5, 5, 5, 2, 5, 2, 2, 5, 5, 2, 5, 2, 2, 5, 5, 2, 5, 2, 2, 2, 2, 5, 2,
2, 5, 2, 2, 2, 5, 5, 2, 5, 5, 2, 5, 2, 5, 2, 5, 5, 5, 5, 5, 5, 5, 2, 2, 5, 5, 2, 2, 2, 5, 2, 2, 2, 5, 2, 5,
2, 2, 2, 5, 5, 2, 5, 5, 2, 5, 2, 5, 2, 5, 2, 2, 5, 5, 5, 5, 2, 5, 2, 5, 5, 5, 5, 2, 2, 5, 5, 2, 2, 5, 2, 5,
2, 5, 5, 2, 2, 5, 5, 5, 5, 5, 5, 5, 2, 2, 5, 5, 5, 5, 2, 2, 5, 2, 5, 2, 5, 2, 5, 5, 5, 5, 2, 5, 2, 5, 5, 2,
2, 5, 5, 5, 2, 2, 5, 2, 2, 5, 2, 5, 5, 2, 5, 5, 5, 2, 2, 2, 2, 2, 2, 5, 5, 2, 2, 2, 2, 5, 5, 2, 5, 5, 5, 5,
5, 5, 5, 2, 5, 2, 2, 5, 2, 2, 2, 2, 5, 2, 5, 2, 5, 2, 2, 2, 2, 2, 2, 5, 2, 2, 5, 5, 5, 2, 2, 2, 2, 5, 2, 5,
2, 2, 2, 5, 2, 5, 2, 5, 2, 5, 2, 2, 5, 5, 5, 5, 5, 5, 2, 5, 5, 2, 2, 2, 5, 5, 5, 2, 2, 2, 2, 2, 2, 5, 2, 5,
2, 2, 5, 5, 2, 2, 2, 2, 2, 5, 5, 5, 2, 5, 2, 2, 2, 5, 5, 2, 2, 2, 2, 2, 2, 5, 5, 2, 5, 2, 5, 2, 2, 5, 5, 2,
2, 2, 5, 2, 5, 5, 5, 5, 2, 5, 2, 5, 2, 5, 5, 2, 5, 2, 5, 5, 2, 2, 5, 2, 2, 5, 5, 2, 5, 2, 5, 2, 2, 5, 5, 5,
5, 5, 2, 5, 5, 2, 2, 5, 2, 2, 5, 5, 5, 2, 5, 5, 5, 2, 5, 5, 5, 2, 2, 5, 5, 5, 5, 2, 5, 5, 2, 2, 2, 5, 2, 5,
2, 5, 2, 2, 5, 5, 2, 2, 5, 5, 2, 2, 2, 5, 5, 5, 2, 2, 5, 5, 5, 2, 2, 5, 2, 2, 2, 2, 5, 2, 2, 5, 5, 2, 2, 5,
2, 2, 2, 5, 5, 2, 5, 5, 2, 5, 2, 5, 2, 5, 5, 5, 2, 2, 5, 2, 5, 5, 5, 2, 5, 5, 5, 2, 2, 2, 5, 2, 2, 2, 5, 2,
```

```python
counter = Counter(result)
counter
```

```
Counter({5: 5045, 2: 4955})
```

```
Drawing Balls from a Bag
5C2 = n!/(n-r)!*r! => 5!/(3!)*2! -> (5*4*3!)/(3!)(2!) => 10 unique ways
{r1,r2,r3,b1,b2}
{r1,r2},{r1,r3},{r1,b1},{r1,b2},{r2,r3}{r2,b1}{r2,b2}{r3,b1}{r3,b2},{b1,b2}
Rearranging the possibilities ....
(X=0) = {b1,b2}
(X=1) = {r1,b1},{r1,b2},{r2,b1},{r2,b2},{r3,b1},{r3,b2}
(X=2) = {r1,r2},{r1,r3},{r2,r3}
```

```
import math
dir(math)
```

```
['__doc__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 'acos',
 'acosh',
 'asin',
 'asinh',
 'atan',
 'atan2',
 'atanh',
 'cbrt',
 'ceil',
 'comb',
 'copysign',
 'cos',
```

```
from math import comb
# Bag [3 red ball & 2 blue ball]
bag = ['R'] * 3 + ['B'] * 2
bag
```

```
['R', 'R', 'R', 'B', 'B']
```

```
trails = 10000
results = []

for _ in range(trails):
    # draw 2 balls without replacement
    draw = random.sample(bag , 2)
    # count how many red balls are picked.
    red_count = draw.count('R')
    results.append(red_count)
```

```
draw
```

```
['R', 'B']
```

```python
print(results)
```

```
1, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 2, 1, 1, 1, 1, 2, 0, 0, 2, 2, 2, 1, 1, 2, 1, 1, 2,
1, 1, 1, 1, 2, 1, 2, 2, 1, 2, 1, 1, 1, 1, 1, 0, 2, 1, 1, 2, 2, 2, 2, 1, 2, 1, 2, 1, 1, 2, 1, 2, 1, 1, 2, 1,
2, 2, 1, 1, 1, 0, 1, 2, 0, 2, 1, 1, 1, 0, 2, 1, 0, 1, 1, 2, 1, 1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 1, 1, 0, 1, 2,
2, 1, 2, 1, 1, 2, 1, 1, 0, 1, 2, 1, 1, 0, 1, 1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 1, 2, 2, 1, 2, 2, 2, 1, 1, 1, 1,
2, 2, 1, 2, 2, 1, 2, 1, 2, 2, 1, 1, 1, 1, 2, 1, 1, 2, 2, 1, 1, 2, 1, 0, 2, 2, 2, 2, 2, 1, 1, 1, 0, 1, 1, 1,
2, 1, 1, 2, 1, 1, 1, 2, 1, 2, 2, 1, 2, 1, 0, 2, 1, 1, 1, 1, 1, 1, 2, 0, 2, 1, 1, 1, 1, 2, 1, 1, 2, 1, 1, 1,
1, 1, 1, 2, 0, 2, 1, 1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 2, 1, 2, 1, 1, 2, 1, 1, 1, 1, 1, 2, 1, 1, 1, 0, 1, 1, 2,
2, 2, 1, 0, 2, 2, 0, 1, 1, 1, 1, 2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 1, 1, 0, 2, 2, 1, 2, 1, 1, 2, 1, 0, 2, 1, 2,
2, 1, 1, 1, 2, 2, 1, 2, 1, 2, 0, 1, 1, 1, 0, 2, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 2, 1,
2, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 2, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 2, 1,
1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 1, 1, 2, 2, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 2, 1, 2, 2, 1, 1, 1, 0, 1, 0, 1, 2,
0, 1, 2, 1, 1, 1, 1, 2, 2, 1, 2, 2, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 0, 1, 1, 0, 1, 1, 1, 2,
2, 2, 1, 0, 1, 1, 1, 1, 1, 1, 2, 2, 1, 1, 0, 2, 1, 2, 0, 1, 2, 2, 0, 2, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 1, 1,
1, 1, 2, 2, 2, 2, 1, 1, 2, 0, 2, 1, 1, 2, 2, 1, 1, 2, 1, 1, 1, 1, 2, 0, 1, 1, 2, 2, 2, 2, 0, 1, 1, 1, 1, 1,
1, 2, 0, 2, 1, 2, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 2, 1, 1, 2, 2, 0, 1, 1, 1, 0, 1, 1,
2, 2, 1, 2, 1, 1, 2, 1, 2, 0, 1, 2, 1, 1, 0, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 2, 1, 1, 1, 2,
1, 2, 1, 1, 1, 1, 1, 2, 2, 1, 1, 2, 1, 1, 1, 1, 1, 2, 1, 2, 2, 1, 1, 1, 0, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 2,
```

```python
counter = Counter(results) # dict
counter
```

```
Counter({1: 6063, 2: 2972, 0: 965})
```

```python
for red_ball_picked , freq in counter.items():
    print(f"Probability of Red ball Picked ({red_ball_picked}) : {freq/trails}")
```

```
Probability of Red ball Picked (1) : 0.6063
Probability of Red ball Picked (2) : 0.2972
Probability of Red ball Picked (0) : 0.0965
```

```python
# Theoretical Distribution using Combinations
total_outcomes = comb(5,2)
total_outcomes
```

```
10
```

```python
# P(X=0) : No Red Ball Picked
p0 = comb(2,2) / total_outcomes # 1 ways / 10
p0
```

```
0.1
```

```python
# P(X=1) : 1 Red Ball Picked
p1 = (comb(3,1) * comb(2,1)) / total_outcomes # 6 ways / 10
p1
```

```
0.6
```

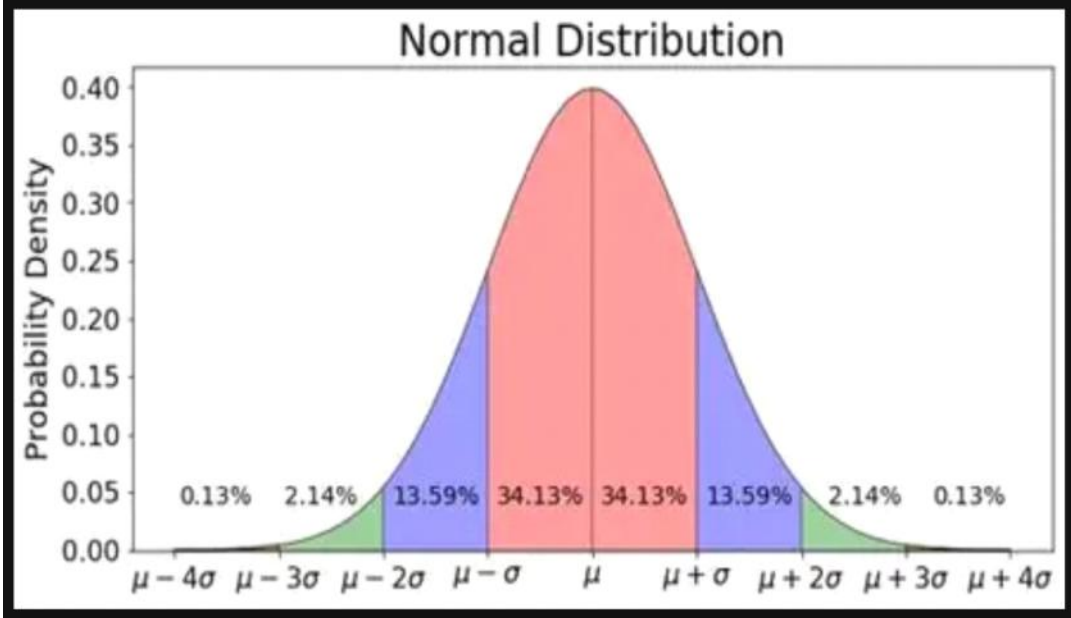
```python
# P(X=2) : 2 Red Ball Picked
p2 = comb(3,2) / total_outcomes # 3 ways / 10
p2
```

```
0.3
```

The Normal Distribution

The normal distribution (Gaussian distribution) is characterized by its bell-shaped curve, defined by mean (μ) and standard deviation (σ).

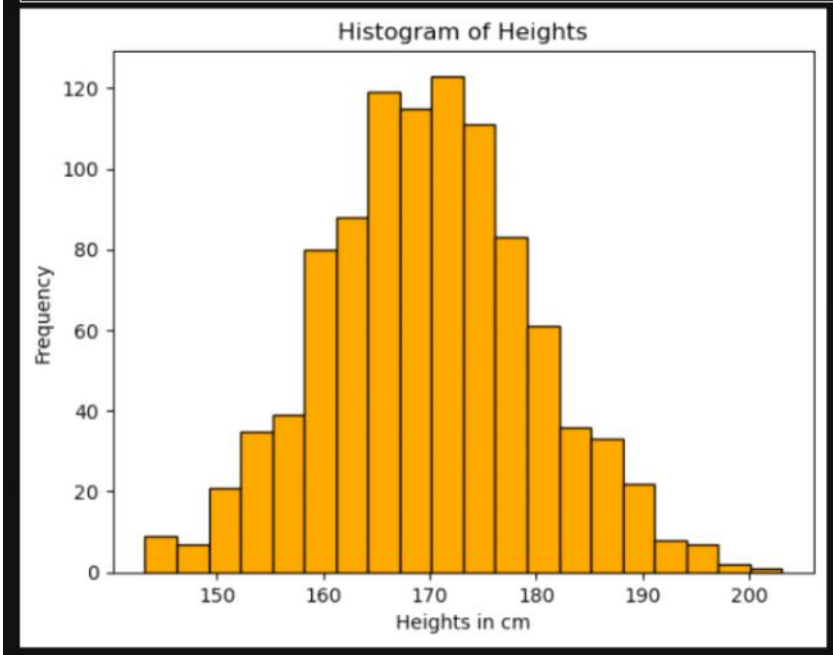
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$




```python
# Distribution in Continuous Variables.
# Fit the normal distribution to your raw data (e.g., 'heights') to automatically calculate
# its optimal mean  and standard deviation

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(77)
heights = np.random.normal(170,10,1000)
heights
```

```
array([172.24444852, 176.61531473, 164.20242361, 174.07811324,
       174.66479367, 150.252909  , 172.75548013, 154.24194827,
       185.68396907, 162.75204977, 173.05889717, 181.63942158,
       159.5090461 , 160.23421994, 163.60949208, 186.97429232,
       157.13884699, 168.10838418, 169.26087332, 168.39504626,
       168.91206523, 163.03015986, 159.94189192, 166.07454951,
       148.62206901, 187.60930893, 162.25806773, 164.69766066,
       162.06483989, 172.91902428, 157.77994639, 154.30962235,
       173.99950159, 169.54355307, 160.84080569, 181.18986476,
       171.9738121 , 172.75060112, 161.2086809 , 159.01760479,
       167.83229509, 159.67935649, 168.3161691 , 175.56938897,
       178.12841843, 168.72887915, 168.47722485, 170.70958812,
       170.19954257, 158.79729891, 152.72796017, 178.8876051 ,
       165.62196016, 163.58389665, 174.27464041, 178.30527518,
       180.8599978 , 168.62362342, 155.93038724, 181.59348124,
       160.08579994, 171.07351098, 164.55208737, 176.17910288,
       199.08049021, 186.69207801, 174.93889637, 167.27774554,
```

```python
plt.hist(heights , bins = 20, color = 'orange' , edgecolor = 'black')
plt.title("Histogram of Heights")
plt.xlabel("Heights in cm")
plt.ylabel("Frequency")
plt.show()
```




```python
# Statistical Library -> Scipy
import scipy.stats
dir(scipy.stats)
```

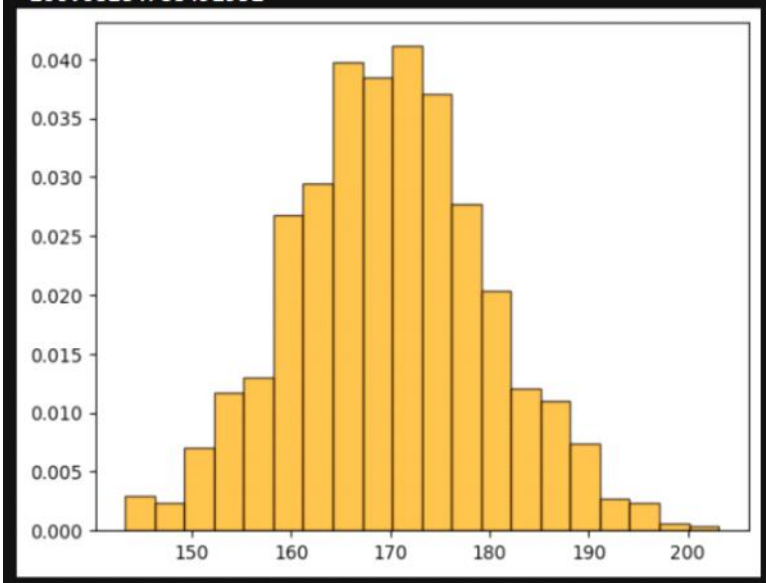
```
 'norm',
 'normaltest',
 'norminvgauss',
 'obrientransform',
 'ortho_group',
 'page_trend_test',
 'pareto',
 'pearson3',
 'pearsonr',
 'percentileofscore',
 'permutation_test',
 'planck',
 'pmean',
 'pointbiserialr',
 'poisson',
 'poisson_means_test',
 'power_divergence',
```

```python
from scipy.stats import norm
mu , std = norm.fit(heights)
print(mu) # mean - 170cm
print(std) # standand_deviation - 10cm
```

```
169.90825878188198
9.893111847089235
```

```
plt.hist(heights , bins = 20 , density = True , alpha = 0.7 , color = 'Orange' , edgecolor = 'black')
xmin , xmax = plt.xlim() # (140,210)
print(xmin)
print(xmax)
```

```
140.24587786054124
206.08284788491932
```




```
x = np.linspace(xmin, xmax, 100)
x
```

```
array([140.24587786, 140.91089776, 141.57591766, 142.24093756,
       142.90595746, 143.57097736, 144.23599726, 144.90101716,
       145.56603705, 146.23105695, 146.89607685, 147.56109675,
       148.22611665, 148.89113655, 149.55615645, 150.22117635,
       150.88619625, 151.55121615, 152.21623605, 152.88125595,
       153.54627585, 154.21129574, 154.87631564, 155.54133554,
       156.20635544, 156.87137534, 157.53639524, 158.20141514,
       158.86643504, 159.53145494, 160.19647484, 160.86149474,
       161.52651464, 162.19153454, 162.85655443, 163.52157433,
       164.18659423, 164.85161413, 165.51663403, 166.18165393,
       166.84667383, 167.51169373, 168.17671363, 168.84173353,
       169.50675343, 170.17177333, 170.83679323, 171.50181312,
       172.16683302, 172.83185292, 173.49687282, 174.16189272,
       174.82691262, 175.49193252, 176.15695242, 176.82197232,
       177.48699222, 178.15201212, 178.81703202, 179.48205192,
       180.14707181, 180.81209171, 181.47711161, 182.14213151,
       182.80715141, 183.47217131, 184.13719121, 184.80221111,
```

```
206.08284788 - 205.41782799
```

```
0.6650198899999964
```

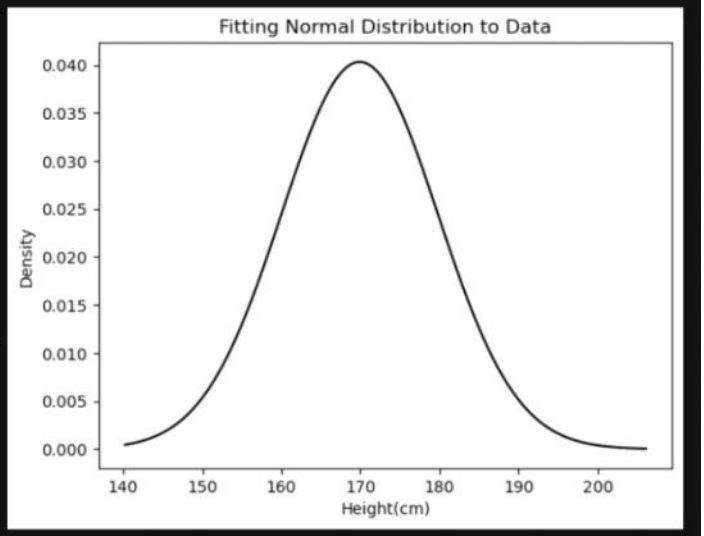
```
140.91089776 - 140.24587786
```

```
0.6650199000000043
```

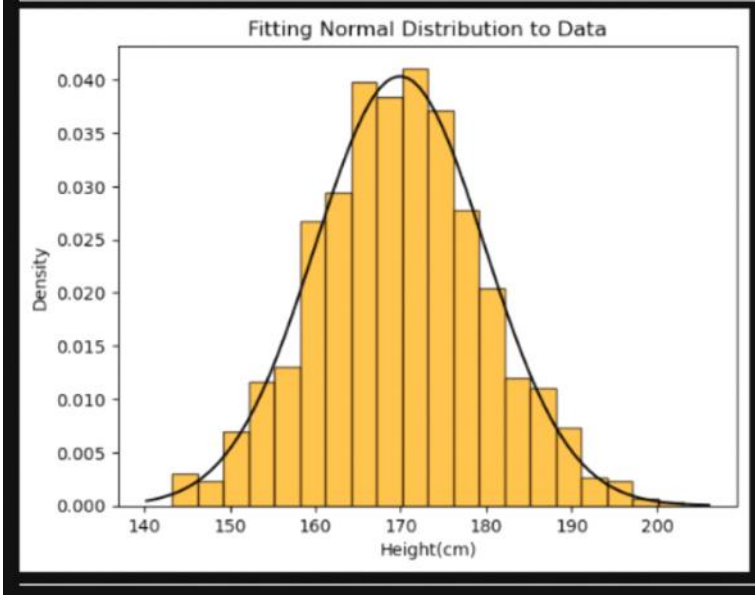
```
dir(norm)
```

```
 'moment',
 'moment_type',
 'name',
 'nnlf',
 'numargs',
 'pdf',
 'ppf',
 'random_state',
 'rvs',
 'sf',
 'shapes',
 'stats',
 'std',
 'support',
 'var',
 'vecentropy',
 'xtol']
```

```
p = norm.pdf(x,mu,std)
plt.plot(x , p , color = 'black')
plt.title('Fitting Normal Distribution to Data')
plt.xlabel('Height(cm)')
plt.ylabel('Density')
plt.show()
```




```
from scipy.stats import norm
mu , std = norm.fit(heights)
# print(mu) # mean - 170cm
# print(std) # standand_deviation - 10cm

plt.hist(heights , bins = 20 , density = True , alpha = 0.7 , color = 'Orange' , edgecolor = 'black')
xmin , xmax = plt.xlim() # (140,210)
# print(xmin) # 140
# print(xmax) # 206

x = np.linspace(xmin, xmax, 100)

p = norm.pdf(x,mu,std)
plt.plot(x , p , color = 'black')
plt.title('Fitting Normal Distribution to Data')
plt.xlabel('Height(cm)')
plt.ylabel('Density')
plt.show()
```

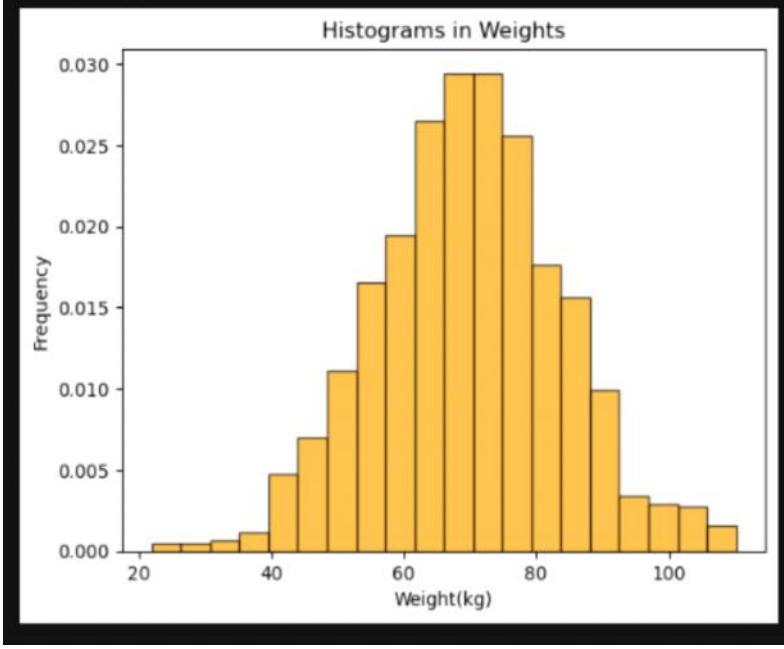


```python
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(10)
weights = np.random.normal(70,15,1000)
weights
```

```
array([ 89.97379756,  80.72918462,  46.81899562,  69.87424225,
        79.32003961,  59.19871659,  73.98267379,  71.62822789,
        70.06437146,  67.38099684,  76.49539285,  88.04556061,
        55.52401494,  85.42411117,  73.42945195,  76.67706419,
        52.95096682,  72.02705318,  92.26805503,  53.80292671,
        40.33407579,  43.84941556,  73.99105246, 105.77450996,
        86.8553688 ,  95.0893332 ,  71.48723824,  90.96994566,
        65.93128018,  79.19806277,  65.99024217,  61.76036479,
        71.99062444,  62.85786978,  89.62709621,  72.92519919,
        76.00314982,  64.93551494,  88.84708396,  59.02045747,
        79.90347327,  64.73692163,  55.9084996 ,  62.65994175,
        57.93113287,  66.80953542,  64.91289631,  74.68254904,
        78.47729004,  67.78869613,  69.61141995,  74.33641306,
        61.90181393,  80.6224003 ,  82.63337108,  73.05371196,
       105.92055497,  83.76188406,  68.31591293,  64.56729329,
        66.51726616,  62.47406651,  86.9317773 ,  59.53284954,
        68.78316724,  62.06055879,  85.69274285,  48.72165956,
```

```python
plt.hist(weights , bins = 20 , density = True , alpha = 0.7 , color = 'Orange' , edgecolor = 'black')
xmin , xmax = plt.xlim() # (16,114)
print(xmin)
print(xmax)

plt.title('Histograms in Weights')
plt.xlabel('Weight(kg)')
plt.ylabel('Frequency')
plt.show()
```

```
17.520746095823835
114.61188835892145
```

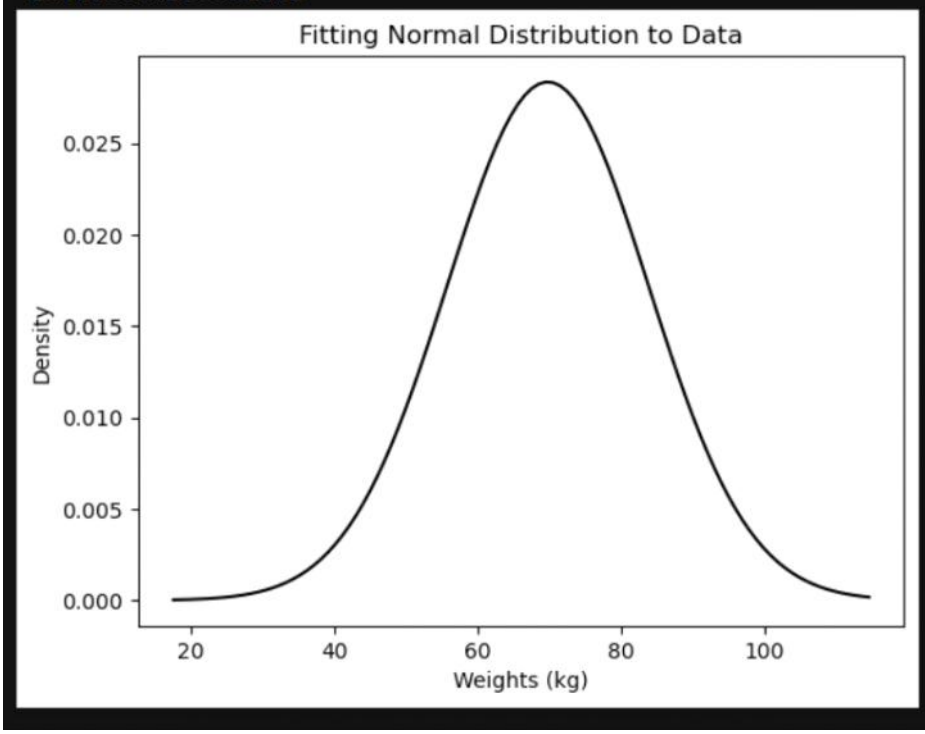


```
x = np.linspace(xmin , xmax , 100)
x
```

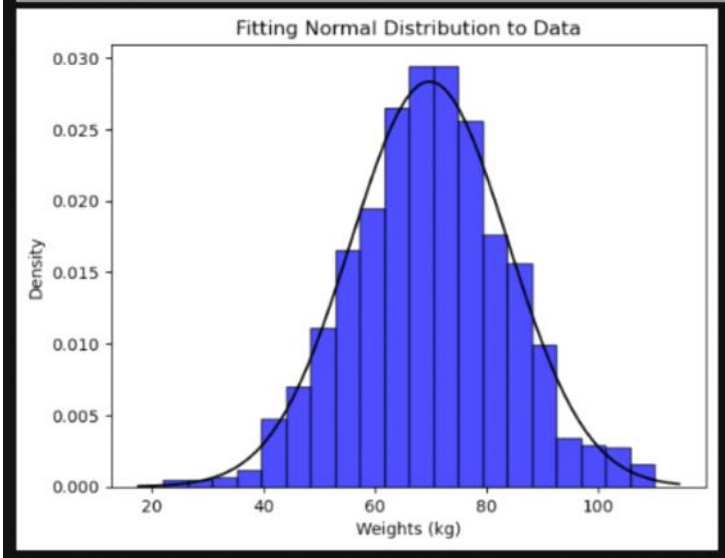
```
array([ 17.5207461 ,  18.5014647 ,  19.48218331,  20.46290192,
        21.44362053,  22.42433914,  23.40505775,  24.38577636,
        25.36649497,  26.34721357,  27.32793218,  28.30865079,
        29.2893694 ,  30.27008801,  31.25080662,  32.23152523,
        33.21224384,  34.19296244,  35.17368105,  36.15439966,
        37.13511827,  38.11583688,  39.09655549,  40.0772741 ,
        41.05799271,  42.03871131,  43.01942992,  44.00014853,
        44.98086714,  45.96158575,  46.94230436,  47.92302297,
        48.90374157,  49.88446018,  50.86517879,  51.8458974 ,
        52.82661601,  53.80733462,  54.78805323,  55.76877184,
        56.74949044,  57.73020905,  58.71092766,  59.69164627,
        60.67236488,  61.65308349,  62.6338021 ,  63.61452071,
        64.59523931,  65.57595792,  66.55667653,  67.53739514,
        68.51811375,  69.49883236,  70.47955097,  71.46026958,
        72.44098818,  73.42170679,  74.4024254 ,  75.38314401,
        76.36386262,  77.34458123,  78.32529984,  79.30601845,
        80.28673705,  81.26745566,  82.24817427,  83.22889288,
```

```
mu , std = norm.fit(weights)
print(mu) # mean - 70kg
print(std) # standand_deviation - 15kg

p = norm.pdf(x,mu,std)
plt.plot(x , p , color = 'black')
plt.title('Fitting Normal Distribution to Data')
plt.xlabel('Weights (kg)')
plt.ylabel('Density')
plt.show()
```

```
69.78165046576795
14.069366357942085
```

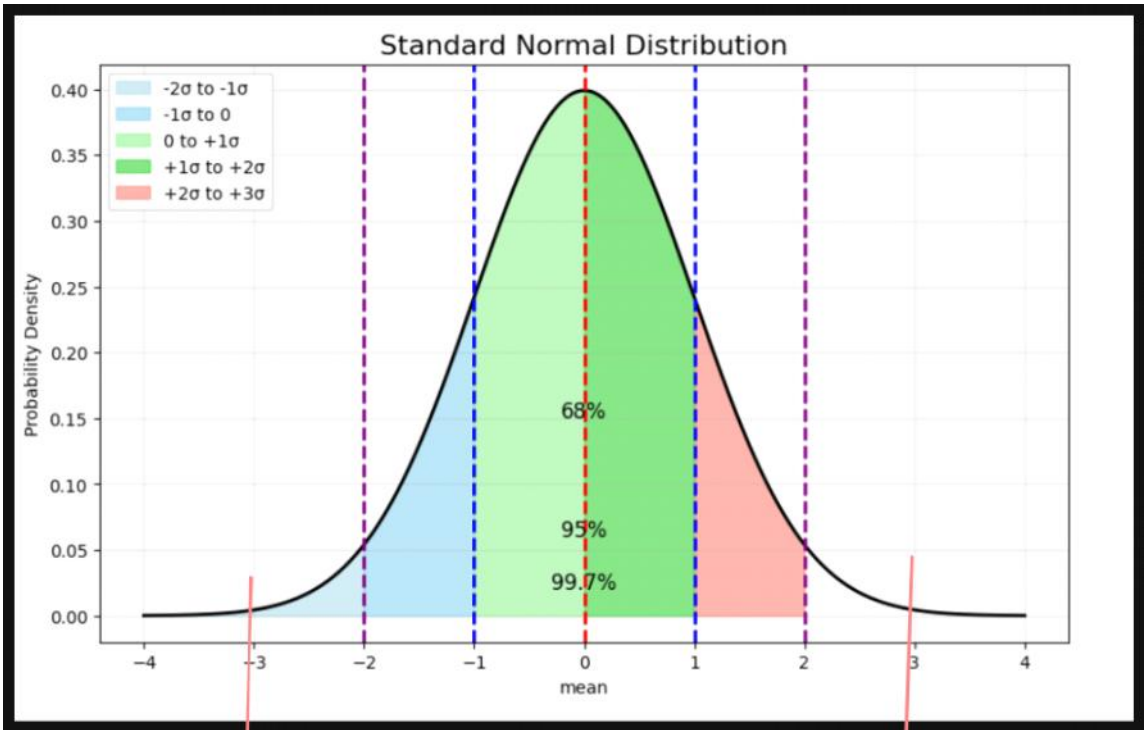


```python
from scipy.stats import norm
mu , std = norm.fit(weights)
# print(mu) # mean - 70kg
# print(std) # standand_deviation - 15kg

plt.hist(weights , bins = 20 , density = True , alpha = 0.7 , color = 'blue' , edgecolor = 'black')
xmin , xmax = plt.xlim() # (16,114)
# print(xmin) # 16
# print(xmax) # 114

x = np.linspace(xmin, xmax, 100)

p = norm.pdf(x,mu,std)
plt.plot(x , p , color = 'black')
plt.title('Fitting Normal Distribution to Data')
plt.xlabel('Weights (kg)')
plt.ylabel('Density')
plt.show()
```




```python
# Standard Normal Distributed Curve
# mean  = 0 and standard deviation = 1
mu = 0
std = 1
x_standard = np.linspace(mu - 3 * std , mu + 3 * std , 100)
x_standard
```

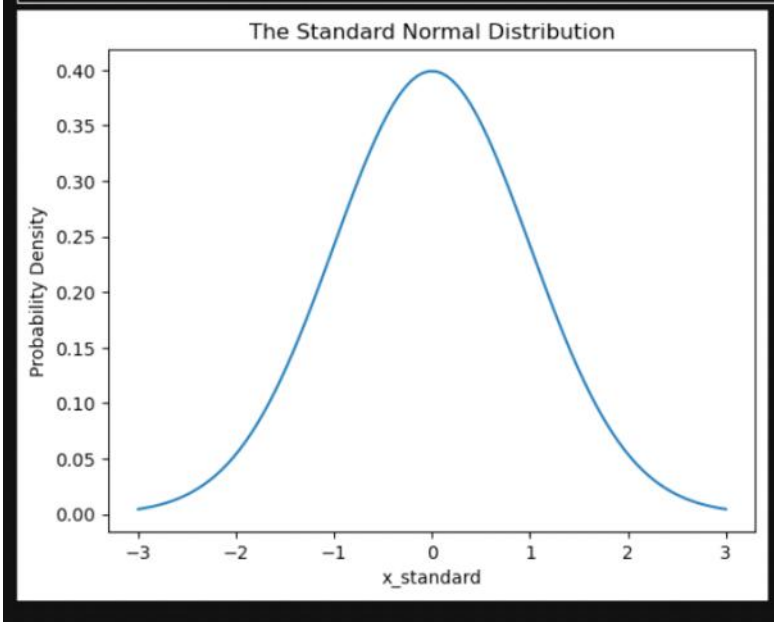
```
array([-3.        , -2.93939394, -2.87878788, -2.81818182, -2.75757576,
       -2.6969697 , -2.63636364, -2.57575758, -2.51515152, -2.45454545,
       -2.39393939, -2.33333333, -2.27272727, -2.21212121, -2.15151515,
       -2.09090909, -2.03030303, -1.96969697, -1.90909091, -1.84848485,
       -1.78787879, -1.72727273, -1.66666667, -1.60606061, -1.54545455,
       -1.48484848, -1.42424242, -1.36363636, -1.3030303 , -1.24242424,
       -1.18181818, -1.12121212, -1.06060606, -1.        , -0.93939394,
       -0.87878788, -0.81818182, -0.75757576, -0.6969697 , -0.63636364,
       -0.57575758, -0.51515152, -0.45454545, -0.39393939, -0.33333333,
       -0.27272727, -0.21212121, -0.15151515, -0.09090909, -0.03030303,
        0.03030303,  0.09090909,  0.15151515,  0.21212121,  0.27272727,
        0.33333333,  0.39393939,  0.45454545,  0.51515152,  0.57575758,
        0.63636364,  0.6969697 ,  0.75757576,  0.81818182,  0.87878788,
        0.93939394,  1.        ,  1.06060606,  1.12121212,  1.18181818,
        1.24242424,  1.3030303 ,  1.36363636,  1.42424242,  1.48484848,
        1.54545455,  1.60606061,  1.66666667,  1.72727273,  1.78787879,
        1.84848485,  1.90909091,  1.96969697,  2.03030303,  2.09090909,
        2.15151515,  2.21212121,  2.27272727,  2.33333333,  2.39393939,
        2.45454545,  2.51515152,  2.57575758,  2.63636364,  2.6969697 ,
        2.75757576,  2.81818182,  2.87878788,  2.93939394,  3.        ])
```



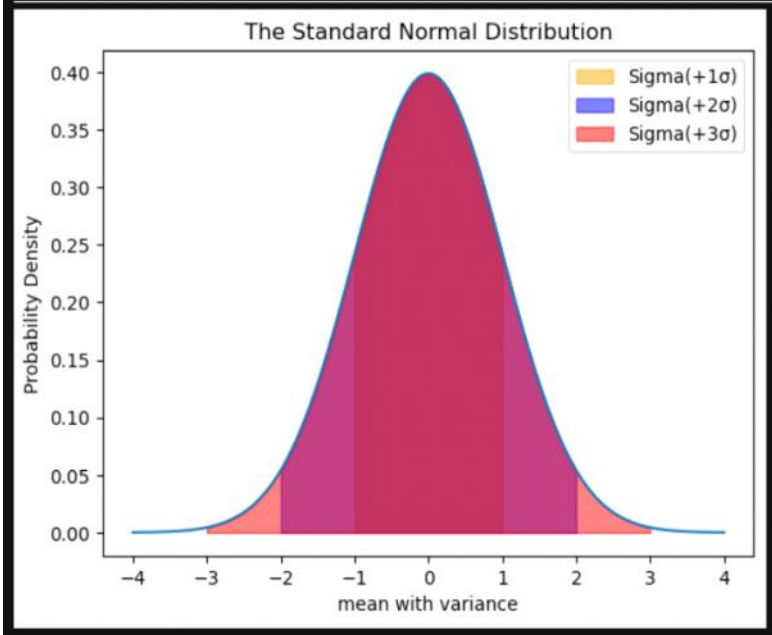
mu - (3 * std)
low

mu + (3 * std)
high

```
# Probability Density  Function
pdf = norm.pdf(x_standard , mu , std)
plt.plot(x_standard , pdf)
plt.title('The Standard Normal Distribution')
plt.xlabel('x_standard')
plt.ylabel('Probability Density')
plt.show()
```



```python
# Standard Normal Distribution showing -> 3 level of confidence interval
x = np.linspace(-4,4,1000)
y = norm.pdf(x,0,1) # mu = 0 , std = 1

plt.plot(x , y)
plt.fill_between(x , y , where = (x>=-1) & (x<=1) , alpha = 0.5 , color = 'orange', label = 'Sigma(+1σ)')
plt.fill_between(x , y , where = (x>=-2) & (x<=2) , alpha = 0.5 , color = 'blue', label = 'Sigma(+2σ)')
plt.fill_between(x , y , where = (x>=-3) & (x<=3) , alpha = 0.5 , color = 'red', label = 'Sigma(+3σ)')
plt.title('The Standard Normal Distribution')
plt.xlabel('mean with variance')
plt.ylabel('Probability Density')
plt.legend()
plt.show()
```




```python
import scipy.stats
dir(scipy.stats.poisson)
```

```
 'median',
 'moment',
 'moment_tol',
 'name',
 'nnlf',
 'numargs',
 'pmf',
 'ppf',
 'random_state',
 'rvs',
 'sf',
 'shapes',
 'stats',
 'std',
 'support',
 'var',
 'vecentropy']
```

```
print(scipy.stats.poisson.__doc__)
```

```
A Poisson discrete random variable.

    As an instance of the `rv_discrete` class, `poisson` object inherits from it
    a collection of generic methods (see below for the full list),
    and completes them with details specific for this particular distribution.

    Methods
    -------
    rvs(mu, loc=0, size=1, random_state=None)
        Random variates.
    pmf(k, mu, loc=0)
        Probability mass function.
    logpmf(k, mu, loc=0)
        Log of the probability mass function.
    cdf(k, mu, loc=0)
        Cumulative distribution function.
    logcdf(k, mu, loc=0)
```

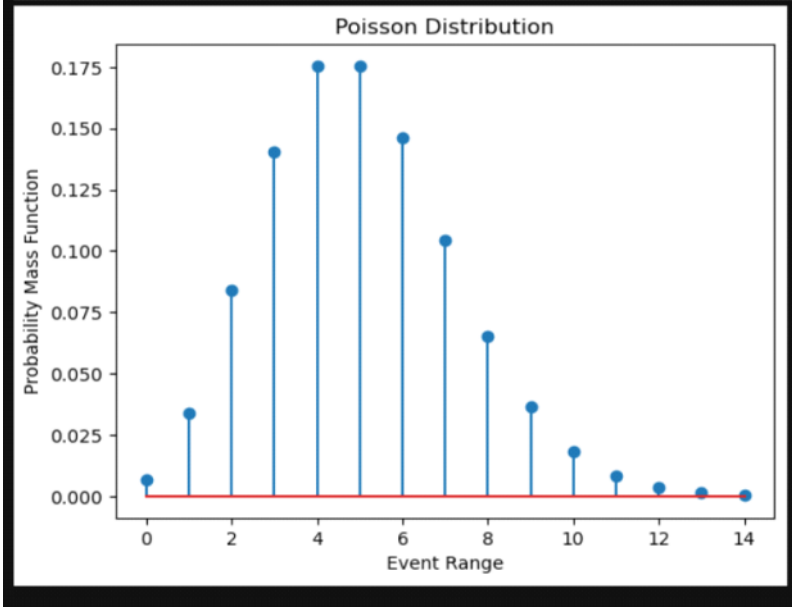
```
from scipy.stats import poisson

Lambda_λ = 5 # discrete -> mu [mean]
event_range = range(0,15) # [0,1,2,3....14] # 'k'
pmf = poisson.pmf(event_range , Lambda_λ)
pmf
```

```
array([0.00673795, 0.03368973, 0.08422434, 0.1403739 , 0.17546737,
       0.17546737, 0.14622281, 0.10444486, 0.06527804, 0.03626558,
       0.01813279, 0.00824218, 0.00343424, 0.00132086, 0.00047174])
```

```
sum(pmf)
```

```
0.9997737463238228
```

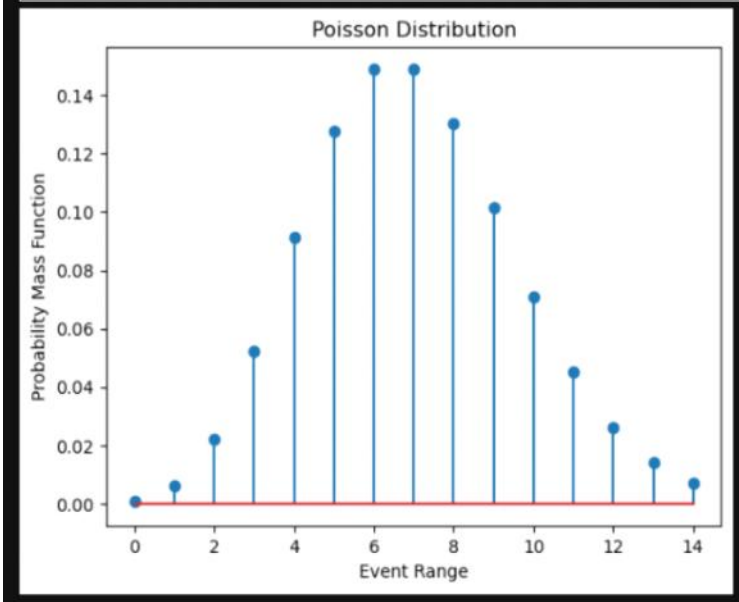
```
# Plotting the Visualization -> stem plot
plt.stem(event_range , pmf)
plt.title('Poisson Distribution')
plt.xlabel('Event Range')
plt.ylabel('Probability Mass Function')
plt.show()
```



```python
from scipy.stats import poisson

Lambda_λ = 7 # discrete -> mu [mean]
event_range = range(0,15) # [0,1,2,3....14] # 'k'
pmf = poisson.pmf(event_range , Lambda_λ)
pmf
```

```
array([0.00091188, 0.00638317, 0.02234111, 0.05212925, 0.09122619,
       0.12771667, 0.14900278, 0.14900278, 0.13037743, 0.10140467,
       0.07098327, 0.04517117, 0.02634985, 0.01418838, 0.00709419])
```

```python
# Plotting the Visualization -> stem plot
plt.stem(event_range , pmf)
plt.title('Poisson Distribution')
plt.xlabel('Event Range')
plt.ylabel('Probability Mass Function')
plt.show()
```



Key Distribution Comparison

Normal Distribution

Type: Continuous

Parameters: μ (mean), σ (std dev)

Use: Natural phenomena, measurements

Shape: Symmetric bell curve

Standard Normal

Type: Continuous

Parameters: μ = 0, σ = 1

Use: Standardization, comparison

Shape: Symmetric bell curve

Poisson Distribution

Type: Discrete

Parameters: λ (rate)

Use: Count of events over time/space

Shape: Right-skewed for small λ